

LAB 2. Manipulasi Data Menggunakan Pandas

1. CPMK dan SubCPMK

CPMK-1018 - Mampu melakukan exploratory data analysis (EDA) dan menerapkan statistik deskriptif untuk analisis data.

Indikator capaian pada pekan ini adalah mahasiswa mampu melakukan pengolahan dan manipulasi data menggunakan tools analitik data untuk menghasilkan dataset yang siap dianalisis.

Sub-CPMK

- 1) Mahasiswa mampu memuat dataset ke dalam DataFrame menggunakan pustaka Pandas.
- 2) Mahasiswa mampu melakukan manipulasi data dasar seperti filtering, sorting, seleksi kolom, serta penanganan data yang tidak lengkap.

2. Dasar Teori

Manipulasi Data

Manipulasi data merupakan proses mengubah, menyusun, atau membersihkan data agar siap digunakan dalam analisis. Data yang diperoleh dari berbagai sumber seringkali belum dalam kondisi yang siap dianalisis. Data dapat memiliki nilai yang hilang, format yang tidak konsisten, atau struktur yang tidak sesuai dengan kebutuhan analisis.

Oleh karena itu, sebelum melakukan analisis statistik atau visualisasi data, diperlukan proses manipulasi data untuk memastikan bahwa data yang digunakan memiliki kualitas yang baik. Manipulasi data dapat meliputi berbagai aktivitas seperti:

- memilih kolom yang relevan
- memfilter data berdasarkan kondisi tertentu
- mengurutkan data
- menangani nilai yang hilang
- mengubah tipe data

Proses ini merupakan salah satu tahap penting dalam analitik data karena kualitas analisis sangat bergantung pada kualitas data yang digunakan.

DataFrame pada Pandas

Pandas merupakan pustaka Python yang digunakan untuk mengolah data tabular. Struktur data utama dalam Pandas adalah DataFrame. DataFrame merupakan struktur data dua dimensi yang menyerupai tabel, dengan baris dan kolom.

Setiap kolom dalam DataFrame dapat memiliki tipe data yang berbeda, seperti numerik, teks, atau tanggal. DataFrame memudahkan pengguna untuk melakukan berbagai operasi manipulasi data seperti seleksi kolom, penyaringan data, agregasi data, serta transformasi data. Sebagai contoh, DataFrame dapat digunakan untuk:

- membaca data dari file CSV atau Excel
- menampilkan sebagian data
- menghitung statistik dasar
- melakukan transformasi data

Karena fleksibilitasnya, Pandas menjadi salah satu pustaka yang paling banyak digunakan dalam kegiatan analitik data.

Membaca Dataset

Langkah pertama dalam manipulasi data adalah membaca dataset ke dalam program. Dalam Python, dataset biasanya dibaca menggunakan fungsi `read_csv()` atau `read_excel()` dari pustaka Pandas.

Contoh membaca dataset CSV:

```
df = pd.read_csv("dataset.csv")
```

Setelah dataset dimuat ke dalam DataFrame, pengguna dapat menampilkan sebagian data menggunakan fungsi `head()` untuk memahami struktur dataset.

Seleksi Kolom

Seleksi kolom dilakukan untuk memilih atribut tertentu yang relevan dengan analisis. Dalam Pandas, kolom dapat dipilih menggunakan nama kolom.

Contoh:

```
df["order_status"]
```

Jika ingin memilih beberapa kolom sekaligus, dapat digunakan daftar nama kolom.

Filtering Data

Filtering data digunakan untuk memilih baris data yang memenuhi kondisi tertentu. Filtering sangat berguna untuk menyeleksi data berdasarkan kriteria tertentu.

Contoh filtering:

```
df[df["order_status"] == "delivered"]
```

Perintah tersebut akan menampilkan data dengan status pesanan yang telah dikirim.

Sorting Data

Sorting digunakan untuk mengurutkan data berdasarkan nilai pada kolom tertentu. Pengurutan dapat dilakukan secara menaik (ascending) atau menurun (descending).

Contoh:

```
df.sort_values("order_purchase_timestamp")
```

Sorting sering digunakan untuk melihat urutan data berdasarkan waktu, nilai transaksi, atau variabel lainnya.

Penanganan Missing Values

Dalam dataset nyata, sering ditemukan nilai yang hilang atau tidak tersedia. Nilai yang hilang biasanya direpresentasikan dengan NaN (Not a Number).

Nilai yang hilang dapat menyebabkan kesalahan dalam analisis sehingga perlu ditangani terlebih dahulu. Penanganan missing values dapat dilakukan dengan beberapa cara, antara lain:

- menghapus baris yang memiliki nilai kosong
- mengganti nilai kosong dengan nilai tertentu
- mengisi nilai kosong dengan rata-rata atau median

Pandas menyediakan beberapa fungsi untuk menangani missing values seperti `dropna()` dan `fillna()`.

3. Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- 1) membaca dataset menggunakan pustaka Pandas
- 2) memahami struktur DataFrame
- 3) menampilkan informasi dasar dataset
- 4) melakukan seleksi kolom
- 5) melakukan filtering data
- 6) mengurutkan data
- 7) menangani nilai yang hilang

4. Dataset

Dataset yang digunakan pada praktikum ini adalah Brazilian E-Commerce Public Dataset.

File yang digunakan:

`olist_orders_dataset.csv`

Dataset ini berisi informasi mengenai transaksi pesanan pelanggan pada sebuah platform e-commerce.

Beberapa kolom penting dalam dataset:

Kolom	Deskripsi
order_id	ID unik pesanan
customer_id	ID pelanggan
order_status	status pesanan
order_purchase_timestamp	waktu pembelian
order_delivered_customer_date	waktu pesanan diterima

Dataset ini digunakan untuk latihan manipulasi data menggunakan Pandas sebelum dilakukan analisis yang lebih kompleks pada praktikum berikutnya.

5. Langkah Praktikum

- 1) Mengimpor Library yang Dibutuhkan

Buka Jupyter Notebook kemudian jalankan kode berikut untuk mengimpor pustaka yang diperlukan.

```
import pandas as pd
import numpy as np
```

Pandas digunakan untuk manipulasi data tabular, sedangkan NumPy digunakan untuk operasi numerik.

- 2) Membaca Dataset

Muat dataset transaksi pesanan ke dalam DataFrame.

```
orders = pd.read_csv("dataset/olist_orders_dataset.csv")
```

Tampilkan beberapa baris pertama untuk melihat struktur awal data.

```
orders.head()
```

3) Menampilkan Dimensi Dataset

Untuk mengetahui ukuran dataset, gunakan perintah berikut:

```
orders.shape
```

Output dari perintah ini akan menampilkan:

- jumlah baris data
- jumlah kolom data

Contoh output:

```
(99441, 8)
```

Artinya dataset memiliki **99.441 baris dan 8 kolom**.

4) Menampilkan Nama Kolom Dataset

Untuk melihat seluruh kolom dalam dataset, gunakan:

```
orders.columns
```

Perintah ini membantu memahami atribut yang tersedia dalam dataset.

5) Melihat Tipe Data Setiap Kolom

Gunakan perintah berikut:

```
orders.dtypes
```

Perintah ini menampilkan tipe data setiap kolom seperti:

- object
- int64
- float64
- datetime

Memahami tipe data penting karena beberapa analisis memerlukan tipe data tertentu.

6) Mengubah Tipe Data Kolom Waktu

Kolom `order_purchase_timestamp` berisi informasi waktu transaksi, namun biasanya terbaca sebagai teks.

Ubah kolom tersebut menjadi tipe datetime.

```
orders["order_purchase_timestamp"] = pd.to_datetime(
    orders["order_purchase_timestamp"]
)
```

Setelah itu periksa kembali tipe data:

```
orders.dtypes
```

7) Membuat Kolom Baru (Feature Engineering Sederhana)

Tambahkan kolom baru yang berisi tahun transaksi.

```
orders["purchase_year"] =
orders["order_purchase_timestamp"].dt.year
```

Tambahkan juga kolom bulan transaksi.

```
orders["purchase_month"] =
orders["order_purchase_timestamp"].dt.month
```

Kolom ini nantinya berguna untuk analisis tren penjualan.

8) Seleksi Kolom Dataset

Pilih beberapa kolom yang dianggap penting untuk analisis awal.

```
orders_subset = orders[
    [
        "order_id",
        "customer_id",
        "order_status",
        "order_purchase_timestamp",
        "purchase_year",
        "purchase_month"
    ]
]
```

Tampilkan hasilnya:

```
orders_subset.head()
```

9) Filtering Data Berdasarkan Kondisi

Tampilkan hanya pesanan dengan status **delivered**.

```
orders_delivered = orders[
    orders["order_status"] == "delivered"
]
```

Lihat jumlah data yang memenuhi kondisi tersebut.

```
orders_delivered.shape
```

10) Filtering dengan Beberapa Kondisi

Misalnya menampilkan pesanan yang:

- statusnya delivered
- terjadi pada tahun 2017

```
orders_filter = orders[
    (orders["order_status"] == "delivered") &
    (orders["purchase_year"] == 2017)
]
```

Tampilkan hasilnya.

```
orders_filter.head()
```

11) Mengurutkan Data

Urutkan transaksi berdasarkan waktu pembelian.

```
orders_sorted = orders.sort_values(
    "order_purchase_timestamp"
)
```

Untuk melihat transaksi terbaru:

```
orders_sorted.tail()
```

12) Menghitung Frekuensi Nilai

Gunakan `value_counts()` untuk mengetahui distribusi status pesanan.

```
orders["order_status"].value_counts()
```

Perintah ini menunjukkan jumlah pesanan untuk setiap status.

Contoh output:

delivered	96478
shipped	1107
canceled	625

13) Menghitung Jumlah Pesanan per Tahun

Gunakan fungsi `groupby()` untuk menghitung jumlah pesanan per tahun.

```
orders.groupby("purchase_year").size()
```

Perintah ini merupakan contoh agregasi sederhana dalam Pandas.

14) Menghitung Jumlah Pesanan per Bulan

Untuk melihat distribusi transaksi per bulan:

```
orders.groupby("purchase_month").size()
```

Output ini dapat digunakan untuk melihat pola transaksi sepanjang tahun.

15) Memeriksa Missing Values

Gunakan perintah berikut:

```
orders.isnull().sum()
```

Perintah ini menampilkan jumlah nilai kosong pada setiap kolom.

16) Menghapus Baris yang Memiliki Missing Value

Metode paling sederhana adalah menghapus baris yang mengandung nilai kosong.

```
df_clean = df.dropna()
```

Metode ini cocok digunakan jika jumlah missing value relatif sedikit dan tidak mempengaruhi keseluruhan dataset.

Kelemahan metode ini adalah dapat menyebabkan kehilangan data yang cukup banyak jika missing value terdapat pada banyak baris.

17) Menghapus Kolom yang Memiliki Banyak Missing Value

Jika sebuah kolom memiliki terlalu banyak nilai kosong, kolom tersebut dapat dihapus dari dataset.

```
df = df.dropna(axis=1)
```

Metode ini digunakan ketika suatu atribut dianggap tidak memiliki kontribusi penting dalam analisis.

18) Mengganti Missing Value dengan Nilai Konstan

Missing value dapat diganti dengan nilai tertentu, misalnya 0 atau nilai default lainnya.

```
df["column_name"] = df["column_name"].fillna(0)
```

Metode ini sering digunakan untuk data numerik ketika nilai kosong dianggap sebagai nilai nol atau tidak ada aktivitas.

19) Mengganti Missing Value dengan Nilai Rata-rata (Mean)

Metode ini umum digunakan pada data numerik.

```
df["column_name"] = df["column_name"].fillna(  
    df["column_name"].mean()  
)
```

Pengisian dengan nilai rata-rata mempertahankan distribusi data secara umum.

Namun metode ini kurang cocok jika dataset memiliki outlier ekstrem.

20) Mengganti Missing Value dengan Median

Median sering digunakan sebagai alternatif mean karena lebih tahan terhadap outlier.

```
df["column_name"] = df["column_name"].fillna(  
    df["column_name"].median()  
)
```

Metode ini cocok untuk data yang memiliki distribusi tidak normal.

21) Mengganti Missing Value dengan Modus

Untuk data kategorikal, missing value dapat diganti dengan nilai yang paling sering muncul.

```
df["column_name"] = df["column_name"].fillna(  
    df["column_name"].mode()[0]  
)
```

Metode ini sering digunakan untuk atribut seperti:

- kategori produk
- jenis pembayaran
- status transaksi

22) Menggunakan Forward Fill

Metode ini digunakan untuk mengisi nilai kosong dengan nilai sebelumnya dalam dataset.

```
df.fillna(method="ffill", inplace=True)
```

Metode ini biasanya digunakan pada data time series.

23) Menggunakan Backward Fill

Metode ini mengisi missing value dengan nilai setelahnya.

```
df.fillna(method="bfill", inplace=True)
```

Metode ini juga umum digunakan pada data berurutan seperti data waktu.

24) Mengganti Missing Value Berdasarkan Kelompok Data

Pada beberapa kasus, missing value dapat diganti berdasarkan rata-rata kelompok tertentu menggunakan groupby.

Contoh:

```
df["price"] =  
df.groupby("product_category")["price"].transform(  
    lambda x: x.fillna(x.mean())  
)
```

Metode ini lebih baik karena mempertimbangkan konteks kelompok data.

Perbandingan Metode Penanganan Missing Value

Metode	Kelebihan	Kekurangan
Menghapus baris	sederhana	kehilangan data
Menghapus kolom	mengurangi noise	kehilangan variabel
Mengisi nilai konstan	mudah diterapkan	dapat mengubah distribusi data
Mean	mempertahankan rata-rata	sensitif terhadap outlier
Median	tahan terhadap outlier	tidak mempertahankan variasi
Mode	cocok untuk data kategorikal	kurang akurat pada distribusi kompleks
Forward fill	cocok untuk time series	tidak selalu logis
Group-based imputation	lebih kontekstual	lebih kompleks

25) Menyimpan Dataset yang Telah Dimodifikasi

Dataset yang telah dimodifikasi dapat disimpan sebagai file baru.

```
orders.to_csv(
    "dataset/orders_processed.csv",
    index=False
)
```

File ini dapat digunakan kembali pada praktikum berikutnya.

Checklist Data Cleaning

Sebelum melakukan analisis data, penting untuk memastikan bahwa dataset berada dalam kondisi yang baik. Data yang tidak bersih dapat menghasilkan analisis yang tidak akurat atau bahkan menyesatkan. Oleh karena itu, dalam proses analitik data biasanya dilakukan tahap data cleaning.

Data cleaning merupakan proses memperbaiki kualitas data dengan cara mengidentifikasi dan memperbaiki kesalahan pada dataset. Proses ini meliputi pemeriksaan struktur data, pengecekan nilai kosong,

identifikasi data duplikat, serta memastikan tipe data yang digunakan sudah sesuai.

Untuk membantu proses tersebut, mahasiswa dapat menggunakan langkah-langkah berikut sebagai checklist pemeriksaan dataset.

1) Memeriksa Struktur Dataset

Langkah pertama adalah memahami struktur dataset yang digunakan. Hal ini mencakup jumlah baris data, jumlah kolom, serta nama kolom yang tersedia.

Gunakan perintah berikut untuk melihat ukuran dataset.

```
df.shape
```

Output dari perintah ini menampilkan jumlah baris dan kolom dalam dataset.

Selanjutnya, tampilkan nama seluruh kolom dataset.

```
df.columns
```

Memahami nama kolom penting untuk menentukan variabel mana yang relevan dengan analisis.

2) Memeriksa Tipe Data

Setiap kolom dalam dataset memiliki tipe data tertentu, misalnya numerik, teks, atau tanggal. Tipe data yang tidak sesuai dapat menyebabkan kesalahan dalam proses analisis.

Gunakan perintah berikut untuk melihat tipe data setiap kolom.

```
df.dtypes
```

Jika diperlukan, tipe data dapat diubah menggunakan fungsi `astype()` atau `to_datetime()`.

Contoh mengubah kolom menjadi tipe tanggal:

```
df["order_purchase_timestamp"] =  
pd.to_datetime(df["order_purchase_timestamp"])
```

3) Memeriksa Missing Values

Missing value merupakan nilai yang tidak tersedia dalam dataset. Nilai ini biasanya ditampilkan sebagai NaN (Not a Number).

Gunakan perintah berikut untuk memeriksa jumlah missing value pada setiap kolom.

```
df.isnull().sum()
```

Jika terdapat missing value, mahasiswa dapat memilih metode penanganan yang sesuai, seperti:

- menghapus baris data
- menghapus kolom
- mengganti dengan mean, median, atau modus

4) Memeriksa Data Duplikat

Data duplikat adalah data yang muncul lebih dari satu kali dalam dataset. Duplikasi data dapat menyebabkan perhitungan statistik menjadi tidak akurat.

Gunakan perintah berikut untuk memeriksa jumlah data duplikat.

```
df.duplicated().sum()
```

Jika ditemukan data duplikat, data tersebut dapat dihapus dengan perintah:

```
df = df.drop_duplicates()
```

5) Memeriksa Distribusi Data

Memahami distribusi nilai dalam dataset membantu mengidentifikasi kemungkinan adanya kesalahan data atau nilai yang tidak wajar.

Gunakan perintah berikut untuk melihat statistik dasar dataset.

```
df.describe()
```

Output dari perintah ini biasanya menampilkan informasi seperti:

- jumlah data
- rata-rata
- nilai minimum
- nilai maksimum
- kuartil data

Statistik ini membantu memahami karakteristik data secara umum.

6) Memeriksa Nilai Unik pada Kolom Kategorikal

Untuk kolom kategorikal, penting untuk mengetahui nilai unik yang terdapat pada kolom tersebut.

Gunakan perintah berikut:

```
df["order_status"].unique()
```

Untuk melihat jumlah masing-masing kategori:

```
df["order_status"].value_counts()
```

Informasi ini membantu memahami distribusi kategori dalam dataset.

7) Membuat Dataset Hasil Pembersihan Data

Setelah proses data cleaning selesai, dataset yang telah diperbaiki dapat disimpan sebagai file baru.

Gunakan perintah berikut:

```
df.to_csv("Dataset/orders_clean.csv", index=False)
```

Dataset yang telah dibersihkan ini dapat digunakan pada tahap analisis berikutnya.

Ringkasan Proses Data Cleaning

Berikut langkah-langkah utama dalam proses data cleaning.

Langkah	Tujuan
Memeriksa struktur data	memahami ukuran dan atribut dataset
Memeriksa tipe data	memastikan format data sesuai
Memeriksa missing values	mengidentifikasi data yang tidak lengkap
Memeriksa duplikasi data	menghindari data yang berulang
Memeriksa distribusi data	memahami karakteristik data
Menyimpan dataset bersih	menyiapkan dataset untuk analisis

Kesalahan Umum dalam Manipulasi Data

Pada proses manipulasi data, sering terjadi kesalahan yang dapat mempengaruhi hasil analisis. Kesalahan tersebut biasanya terjadi karena kurangnya pemahaman terhadap struktur dataset atau kesalahan dalam penggunaan perintah pada pustaka Pandas. Oleh karena itu, mahasiswa perlu memahami beberapa kesalahan umum yang sering terjadi dalam proses manipulasi data.

1) Tidak Memeriksa Struktur Dataset

Salah satu kesalahan yang paling sering terjadi adalah langsung melakukan analisis tanpa terlebih dahulu memahami struktur dataset.

Mahasiswa sering kali langsung melakukan perhitungan atau visualisasi tanpa mengetahui:

- jumlah data yang tersedia
- jumlah kolom dataset

- tipe data setiap kolom

Akibatnya, analisis yang dilakukan dapat menghasilkan kesimpulan yang tidak tepat.

Untuk menghindari kesalahan tersebut, dataset sebaiknya diperiksa terlebih dahulu

2) Tidak Memperhatikan Tipe Data

Kesalahan lain yang sering terjadi adalah tidak memperhatikan tipe data setiap kolom. Beberapa operasi analisis memerlukan tipe data tertentu agar dapat dijalankan dengan benar.

Sebagai contoh, kolom yang berisi tanggal sering kali terbaca sebagai teks. Jika tipe data tersebut tidak diubah menjadi tipe `datetime`, maka analisis waktu tidak dapat dilakukan.

Memastikan tipe data yang benar merupakan langkah penting dalam manipulasi data.

3) Menghapus Terlalu Banyak Data

Ketika menemukan `missing value`, sebagian mahasiswa langsung menghapus seluruh baris data yang memiliki nilai kosong. Pendekatan ini dapat menyebabkan hilangnya sebagian besar data dalam dataset.

Meskipun metode ini sederhana, penggunaannya perlu dilakukan dengan hati-hati. Jika jumlah `missing value` cukup besar, maka metode lain seperti pengisian nilai rata-rata atau median dapat menjadi alternatif yang lebih baik.

4) Salah Menggunakan Operator Logika

Kesalahan juga sering terjadi ketika melakukan filtering data menggunakan beberapa kondisi sekaligus. Dalam `Pandas`, operator logika harus ditulis dengan benar.

Jika tanda kurung tidak digunakan dengan benar, maka kode dapat menghasilkan `error` atau hasil yang tidak sesuai.

5) Tidak Menyimpan Dataset yang Telah Dimodifikasi

Setelah melakukan manipulasi data, dataset yang telah dimodifikasi sebaiknya disimpan kembali sebagai file baru. Jika dataset tidak disimpan, seluruh perubahan dapat hilang ketika notebook ditutup. File ini dapat digunakan kembali pada praktikum berikutnya sehingga mahasiswa tidak perlu mengulangi proses manipulasi data dari awal.

6) Tidak Mendokumentasikan Proses Analisis

Dalam analitik data, dokumentasi proses analisis sangat penting. Kesalahan yang sering terjadi adalah mahasiswa hanya menuliskan kode program tanpa memberikan penjelasan mengenai langkah-langkah yang dilakukan.

Dalam Jupyter Notebook, dokumentasi dapat ditambahkan menggunakan Markdown Cell. Dengan cara ini, proses analisis dapat dijelaskan secara sistematis dan mudah dipahami.

6. Hasil yang Diharapkan

Setelah menyelesaikan langkah-langkah praktikum, mahasiswa diharapkan memperoleh pemahaman mengenai proses manipulasi data menggunakan pustaka Pandas. Mahasiswa juga diharapkan mampu mengidentifikasi struktur dataset, melakukan transformasi data, serta menghasilkan dataset yang lebih siap digunakan untuk analisis.

Hasil praktikum yang diharapkan antara lain:

- 1) Dataset berhasil dimuat ke dalam DataFrame menggunakan pustaka Pandas.
- 2) Mahasiswa mampu menampilkan struktur dataset menggunakan fungsi seperti `head()`, `info()`, dan `describe()`.
- 3) Mahasiswa mampu melakukan seleksi kolom dan filtering data berdasarkan kondisi tertentu.
- 4) Mahasiswa mampu mengubah tipe data pada kolom yang diperlukan.
- 5) Mahasiswa mampu membuat kolom baru dari variabel yang sudah ada.
- 6) Mahasiswa mampu melakukan agregasi data menggunakan fungsi seperti `groupby()`.

- 7) Mahasiswa mampu memeriksa missing value dan data duplikat pada dataset.
- 8) Mahasiswa mampu menyimpan dataset hasil manipulasi ke dalam file baru.

Dataset hasil manipulasi ini akan digunakan kembali pada praktikum berikutnya untuk melakukan analisis statistik deskriptif dan eksplorasi data.

7. Soal Praktikum

- 1) Apa yang dimaksud dengan manipulasi data dalam analitik data?
- 2) Jelaskan fungsi pustaka Pandas dalam Python.
- 3) Apa perbedaan antara fungsi head() dan info() pada Pandas?
- 4) Bagaimana cara memilih beberapa kolom tertentu dari sebuah DataFrame?
- 5) Jelaskan fungsi filtering data dalam manipulasi data.
- 6) Bagaimana cara mengubah tipe data pada sebuah kolom dalam Pandas?
- 7) Jelaskan tujuan penggunaan fungsi groupby() dalam analisis data.
- 8) Mengapa missing value perlu diperiksa sebelum melakukan analisis data?
- 9) Apa yang dimaksud dengan data duplikat dalam dataset?
- 10) Mengapa dataset hasil manipulasi sebaiknya disimpan sebagai file baru?

8. Tugas Laporan

Mahasiswa diminta menyusun laporan praktikum berdasarkan kegiatan yang telah dilakukan pada Lab 2.

- Laporan harus memuat bagian berikut:
- Judul Praktikum
- Tujuan Praktikum
- Dasar Teori
- Dataset yang digunakan
- Langkah Praktikum
- Hasil Praktikum

- Pembahasan
- Kesimpulan
- Lampiran

Lampiran laporan harus berisi:

- screenshot hasil pembacaan dataset (`head()`)
- screenshot informasi dataset (`info()`)
- screenshot hasil filtering data
- screenshot hasil agregasi data
- screenshot pemeriksaan missing value